



Scheduling

Credits

Sistemi Operativi

Modulo 5: Scheduling

Renzo Davoli
Alberto Montresor

Copyright © 2002-2005 Renzo Davoli, Alberto Montresor

Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.2 or any later version published by the Free Software Foundation; with no Invariant Sections, no Front-Cover Texts, and no Back-Cover Texts. A copy of the license can be found at:

<http://www.gnu.org/licenses/fdl.html#TOC1>

Sezione 1



1. Scheduler, processi e thread

Introduzione

- **Un sistema operativo è un gestore di risorse**
 - processore, memoria principale e secondaria, dispositivi
 - **Per svolgere i suoi compiti, un sistema operativo ha bisogno di strutture dati per mantenere informazioni sulle risorse gestite**
 - **Queste strutture dati comprendono:**
 - tabelle di memoria
 - tabelle di I/O
 - tabelle del file system
 - tabelle dei processi
- **Argomento di questo modulo**

Introduzione

- **Tabelle per la gestione della memoria**

- allocazione memoria per il sistema operativo
- allocazione memoria principale e secondaria per i processi
- informazioni per i meccanismi di protezione

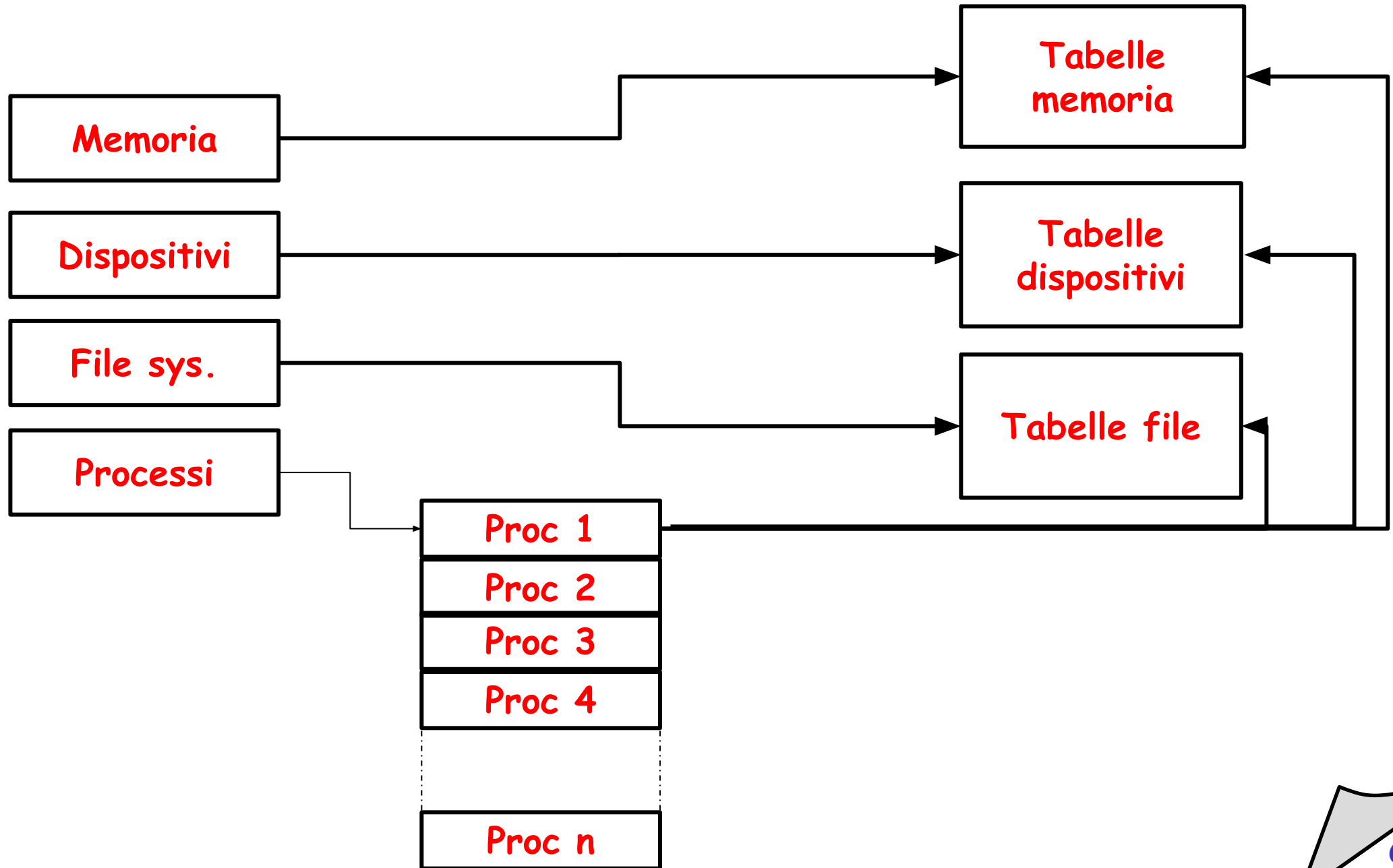
- **Tabelle per la gestione dell'I/O**

- informazioni sullo stato di assegnazione dei dispositivi utilizzati dalla macchina
- gestione di code di richieste

- **Tabelle per la gestione del file system**

- elenco dei dispositivi utilizzati per mantenere il file system
- elenco dei file aperti e loro stato

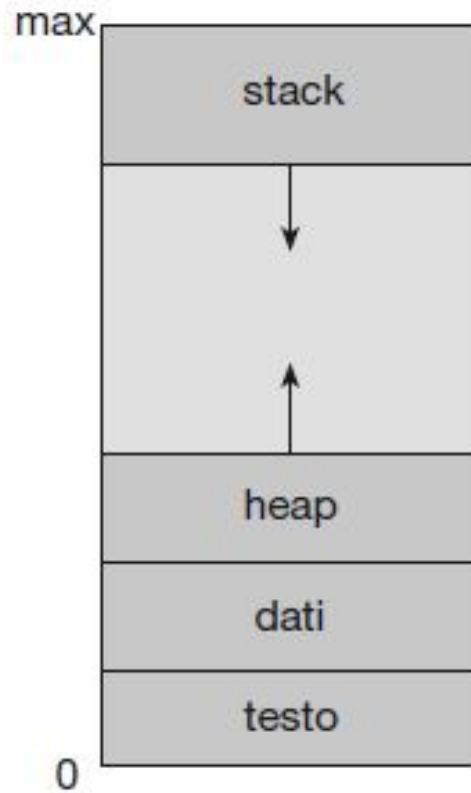
Introduzione



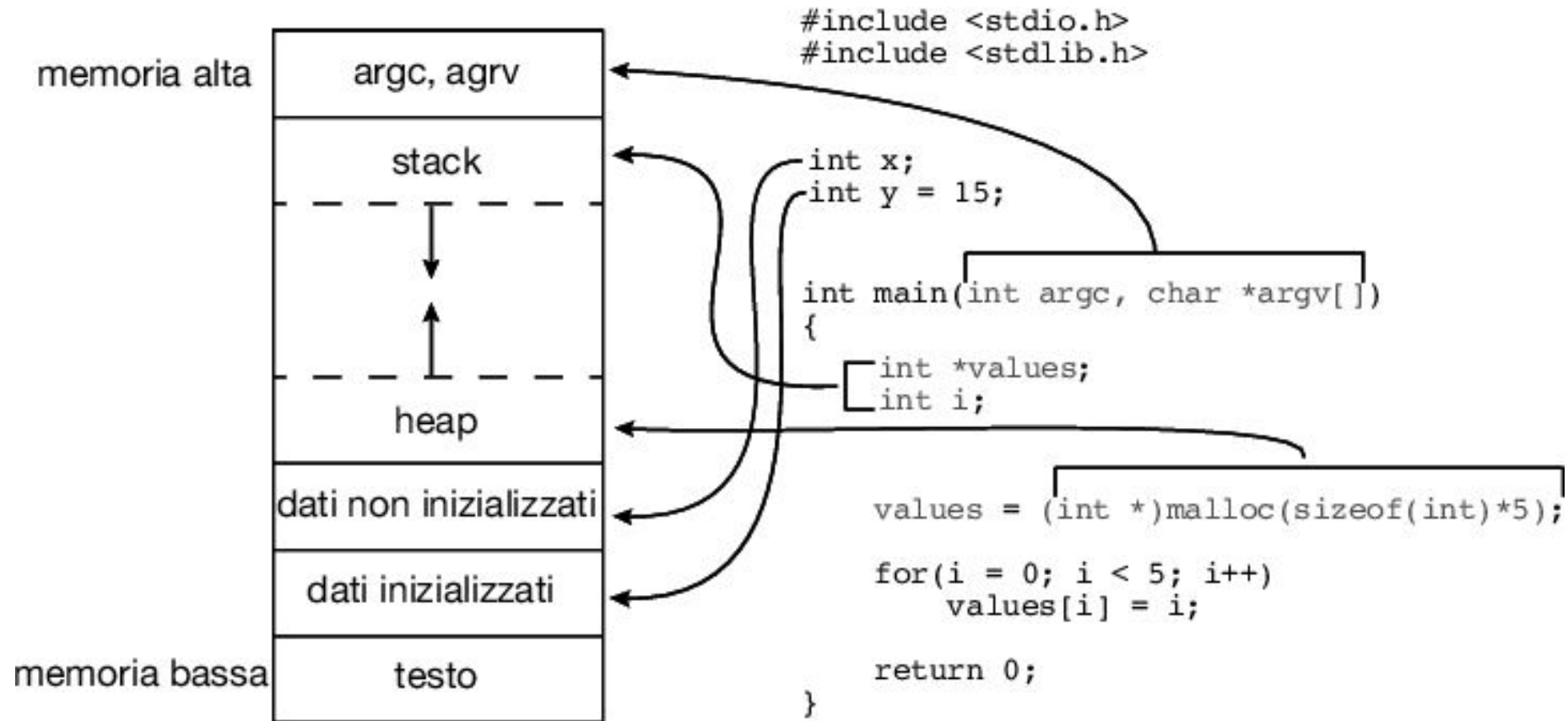
Descrittori dei processi

- Qual é la manifestazione fisica di un processo?
 - a. il codice da eseguire (segmento codice)
 - b. i dati su cui operare (segmenti dati)
 - c. uno stack di lavoro per la gestione di chiamate di funzione, passaggio di parametri e variabili locali
 - d. un *insieme di attributi* contenenti tutte le informazioni necessarie per la gestione del processo stesso
 - incluse le informazioni necessarie per descrivere i punti a-c
- Questo insieme di attributi prende il nome di ***descrittore del processo (process control block, PCB)***

Processo in memoria



Processo in memoria



- vedi **esempio_allocazione.c**

Descrittori dei processi

- **Tabella per la gestione dei processi**
 - contiene i descrittori dei processi
 - ogni processo ha un descrittore associato
- **E' possibile suddividere le informazioni contenute nel descrittore in tre aree:**
 - informazioni di identificazione di processo
 - informazioni di stato del processo
 - informazioni di controllo del processo

Descrittori dei processi

- **Informazioni di identificazione di un processo**

- *identificatore di processo (process id, o pid)*
 - può essere semplicemente un *indice* all'interno di una tabella di processi
 - può essere un *numero progressivo*; in caso, è necessario un mapping tra pid e posizione del relativo descrittore
 - molte altre tabelle del s.o. utilizzano il process id per identificare un elemento della tabella dei processi
- *identificatori di altri processi logicamente collegati al processo*
 - ad esempio, pid del processo padre
- *id dell'utente che ha richiesto l'esecuzione del processo*

Descrittori dei processi

- **Informazioni di stato del processo**

- *registri generali del processore*
- *registri speciali, come il program counter e i registri di stato*

- **Informazioni di controllo del processo**

- *Informazioni di scheduling*

- stato del processo

- in esecuzione, pronto, in attesa

- informazioni particolari necessarie dal particolare algoritmo di scheduling utilizzato

- priorità, puntatori per la gestione delle code

- identificatore dell'evento per cui il processo è in attesa

Descrittori dei processi

- **Informazioni di controllo del processo (continua)**

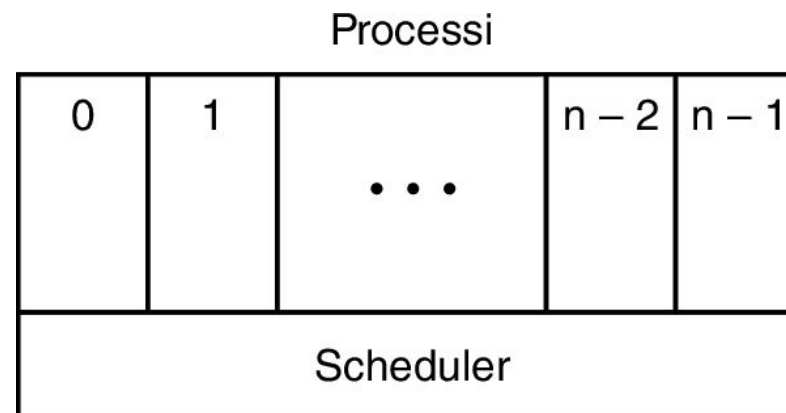
- informazioni di gestione della memoria
 - valori dei registri base e limite dei segmenti utilizzati, puntatori alle tabelle delle pagine, etc.
- informazioni di accounting
 - tempo di esecuzione del processo
 - tempo trascorso dall'attivazione di un processo
- informazioni relative alle risorse
 - risorse controllate dal processo, come file aperti, device allocati al processo
- informazioni per interprocess communication (IPC)
 - stato di segnali, semafori, etc.

Descrittori dei processi

- Vedi `process-descriptor-info.c`

Scheduler

- E' la componente più importante del kernel
- Gestisce l'avvicendamento dei processi
 - decide quale processo deve essere in esecuzione ad ogni istante
 - interviene quando viene richiesta un'operazione di I/O e quando un'operazione di I/O termina, ma anche periodicamente
- NB
 - Il termine "scheduler" viene utilizzato anche in altri ambiti con il significato di "gestore dell'avvicendamento del controllo"
 - possiamo quindi fare riferimento allo "scheduler del disco", e in generale allo "scheduler del dispositivo X"



Schedule, scheduling, scheduler

- **Schedule**

- è la sequenza temporale di assegnazioni delle risorse da gestire ai richiedenti

- **Scheduling**

- è l'azione di calcolare uno schedule

- **Scheduler**

- è la componente software che calcola lo schedule

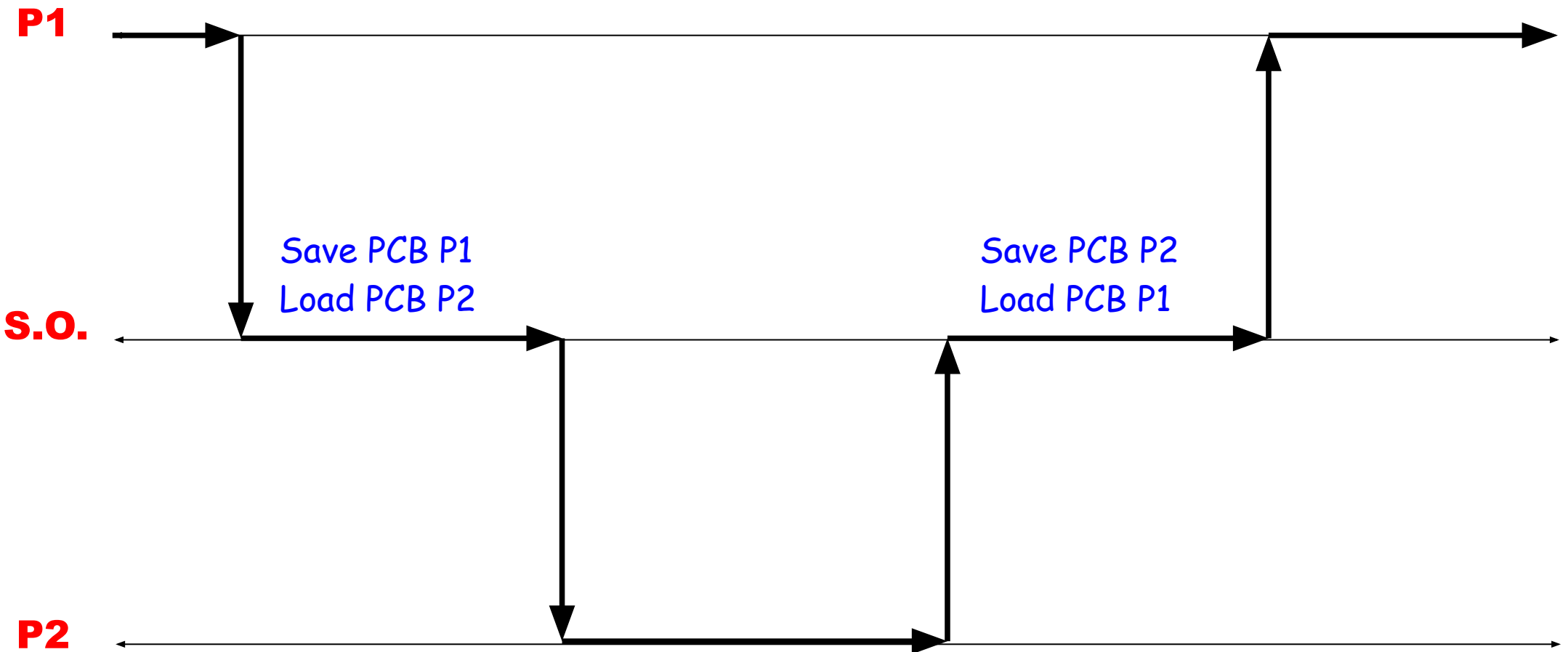
Mode switching e context switching

- Tutte le volte che avviene un interrupt (software o hardware) il processore è soggetto ad un **mode switching**
 - modalità utente → modalità supervisore
- Durante la gestione dell'interrupt
 - vengono intraprese le opportune azioni per gestire l'evento
 - viene chiamato lo scheduler
 - se lo scheduler decide di eseguire un altro processo, il sistema è soggetto ad un **context switching**

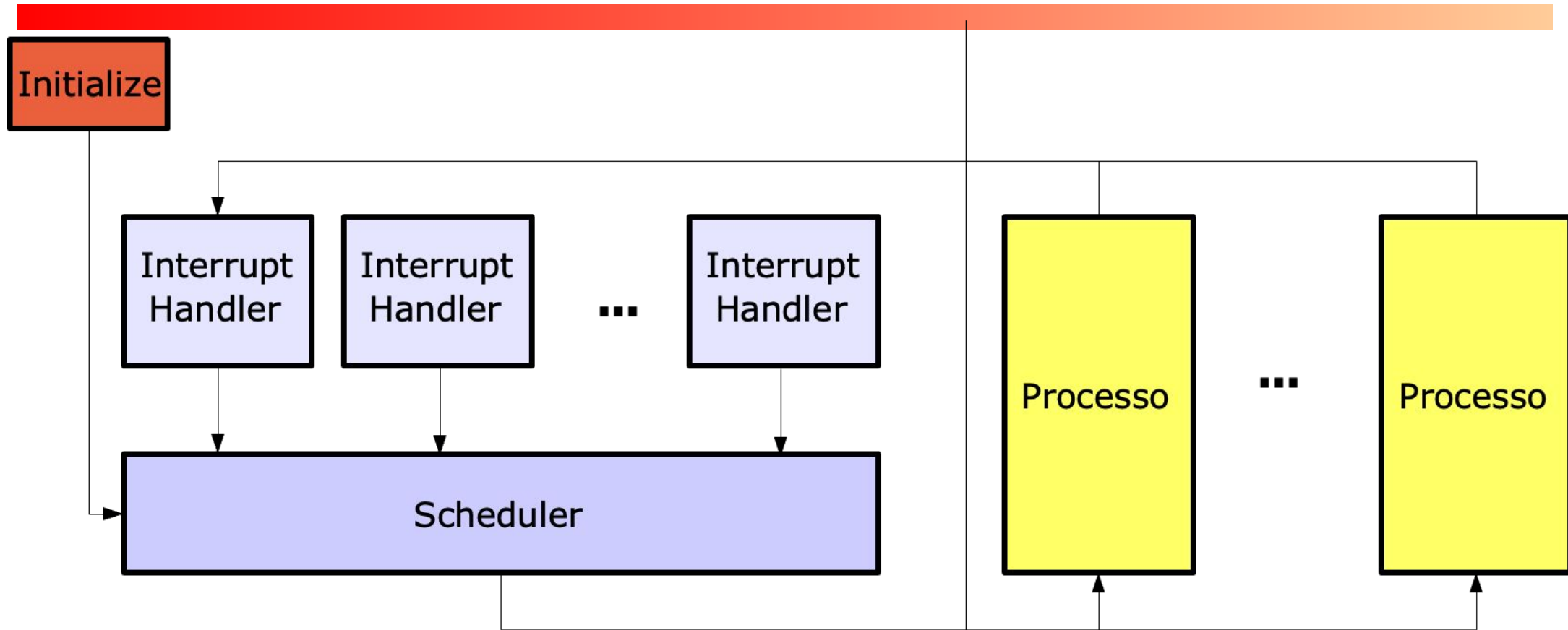
Context switching

- **Operazioni durante un context switching**
 - lo stato del processo attuale viene salvato nel PCB corrispondente
 - lo stato del processo selezionato per l'esecuzione viene caricato dal PCB nel processore

Context switch



Schema di funzionamento di un kernel



Esempio

- Vedi `simulazione_interrupt.c`
 - Il programma inizia in "modalità utente"
 - Usa `signal()` per impostare un gestore per l'interrupt `SIGINT` (Ctrl+C)
 - In un ciclo infinito, simula il lavoro stampando 'U' (utente) o 'K' (kernel)
 - Quando l'utente preme Ctrl+C, viene chiamato `interrupt_handler()`:
 - Entra in "modalità kernel"
 - Simula il lavoro di gestione dell'interrupt
 - Torna in "modalità utente"

Esempio - Approfondimento sul codice

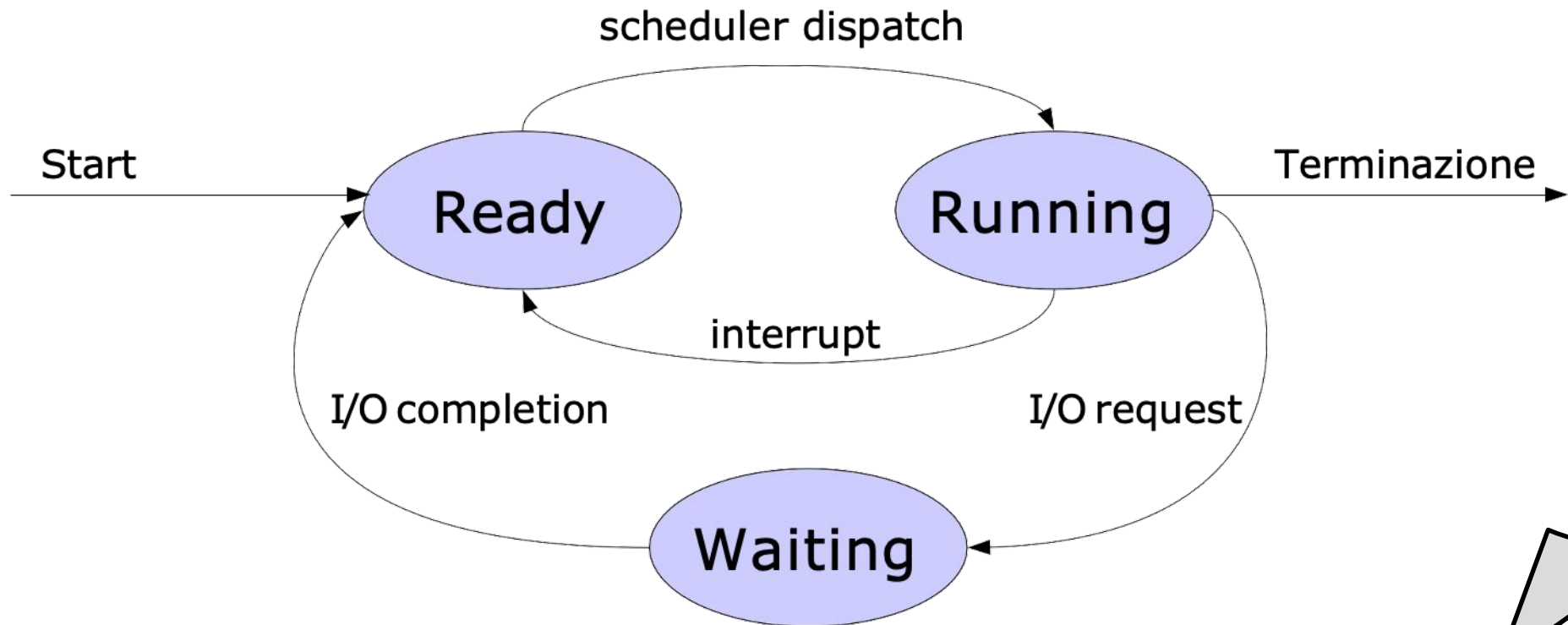
- Vedi `simulazione_interrupt.c`

- variable `in_kernel_mode` è dichiarata `volatile`
- La ragione è legata al modo in cui il compilatore ottimizza il codice e come i segnali interagiscono con il programma
 - **Ottimizzazioni del compilatore:** Senza ``volatile``, il compilatore potrebbe ottimizzare il codice assumendo che ``in_kernel_mode`` non cambi (no assegnamenti nel loop)
 - **Evitare il caching in registri:** Senza ``volatile``, il compilatore potrebbe decidere di caricare il valore di ``in_kernel_mode`` in un registro all'inizio del ciclo e non ricaricarlo ad ogni iterazione, perdendo così eventuali modifiche fatte dal gestore di segnali.
 - **Modifiche esterne al flusso normale:** ``in_kernel_mode`` viene modificata all'interno del gestore di segnali (``interrupt_handler``), che è una routine esterna al flusso di esecuzione normale del programma

Vita di un processo

- **Stati dei processi:**

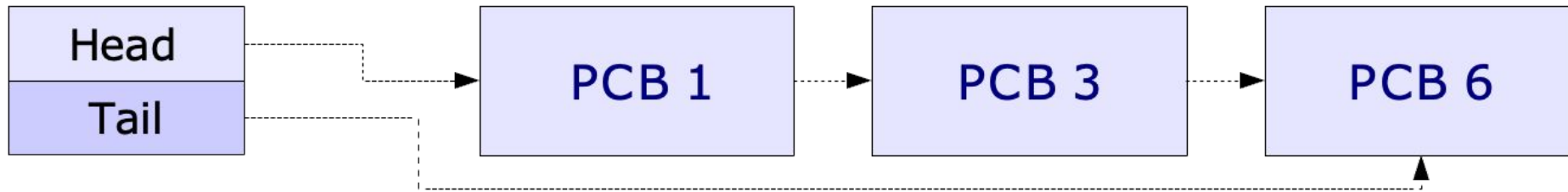
- *Running*: il processo è in esecuzione
- *Waiting*: il processo è in attesa di qualche evento esterno (e.g., completamento operazione di I/O); non può essere eseguito
- *Ready*: il processo può essere eseguito, ma attualmente il processore è impegnato in altre attività



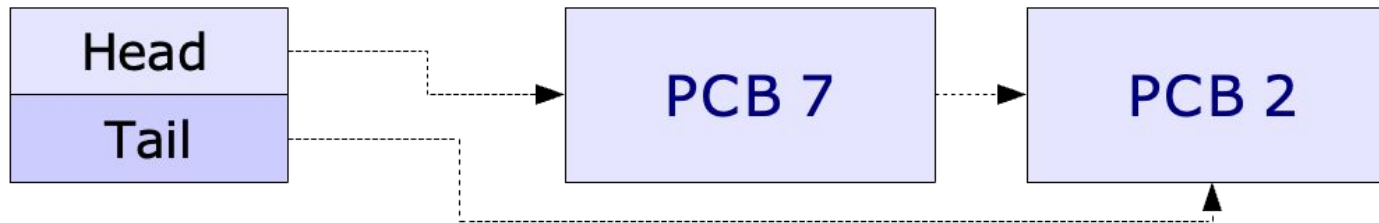
Code di processi

Tutte le volte che un processo entra nel sistema, viene posto in una delle code gestite dallo scheduler

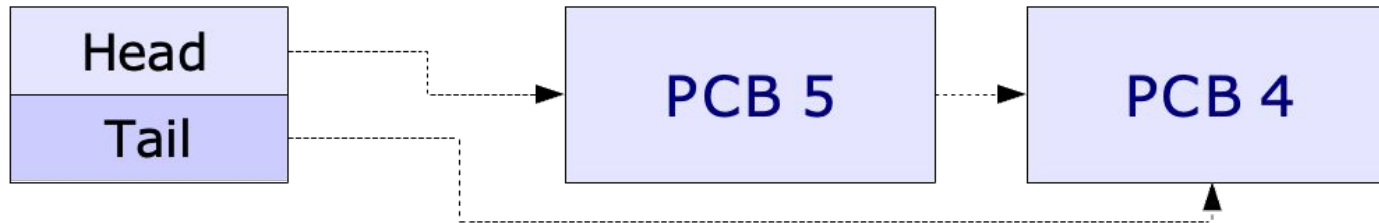
Ready
Queue



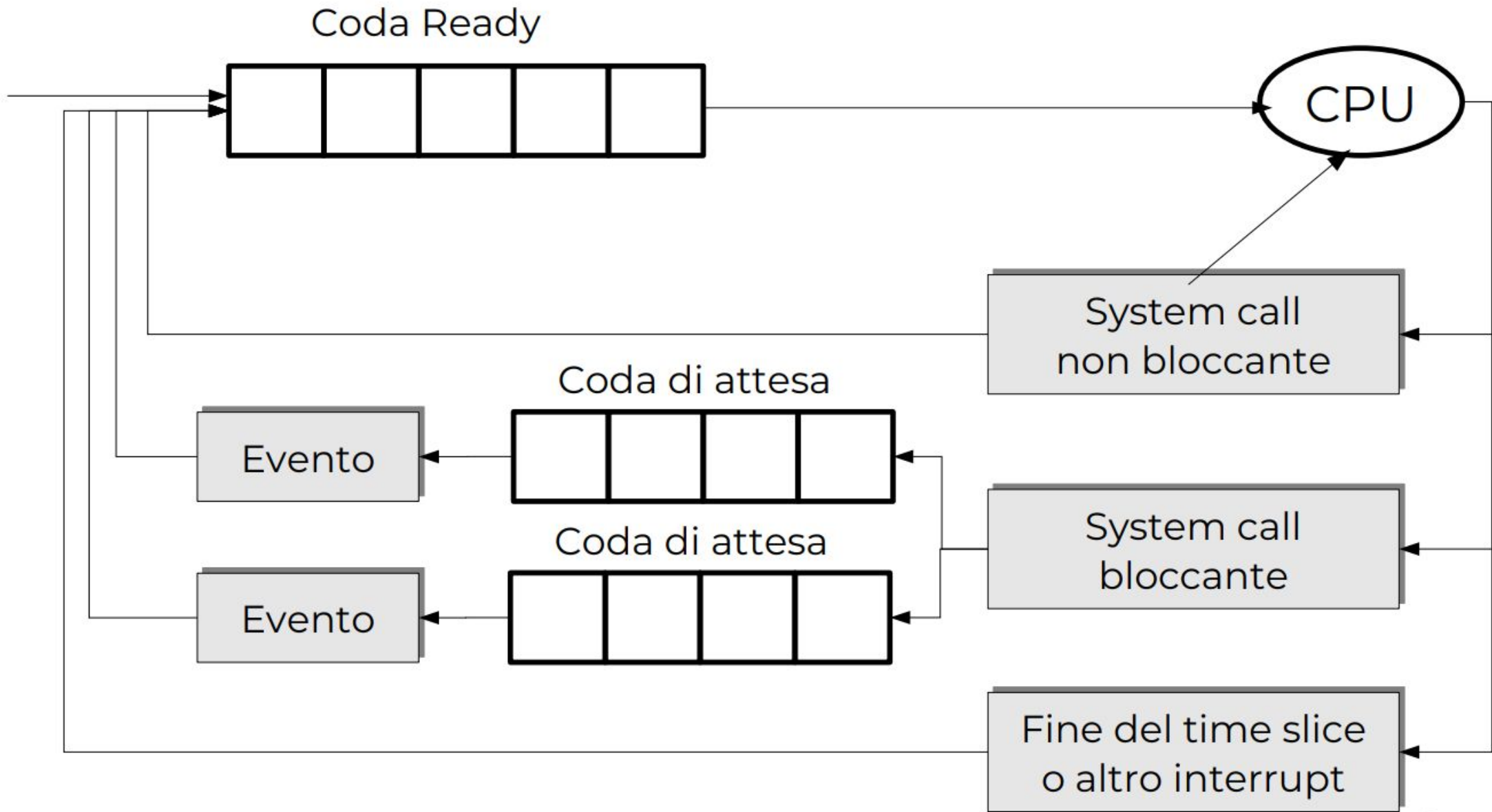
Disk
Queue



Terminal
Queue



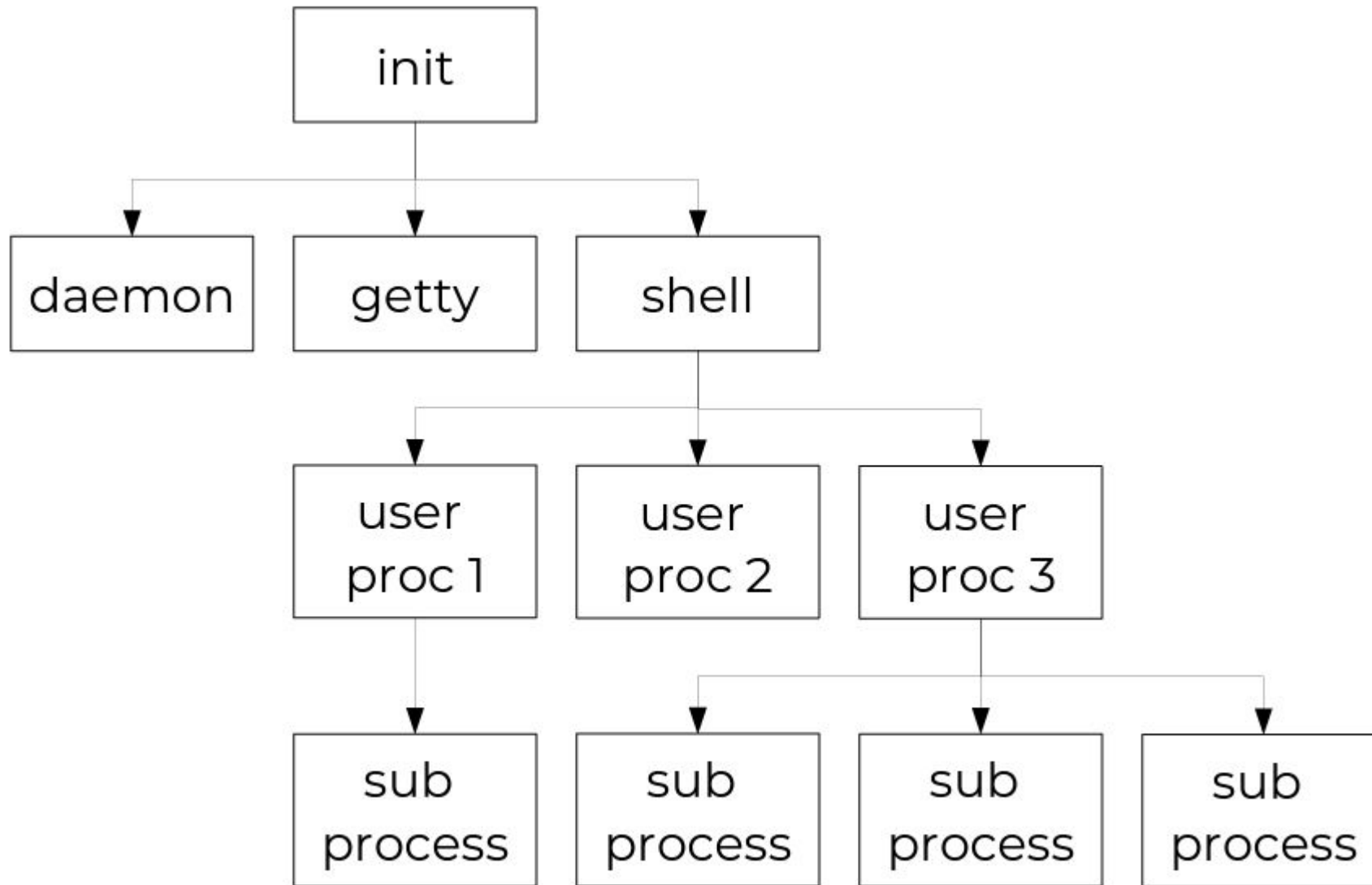
Vita di un processo nello scheduler



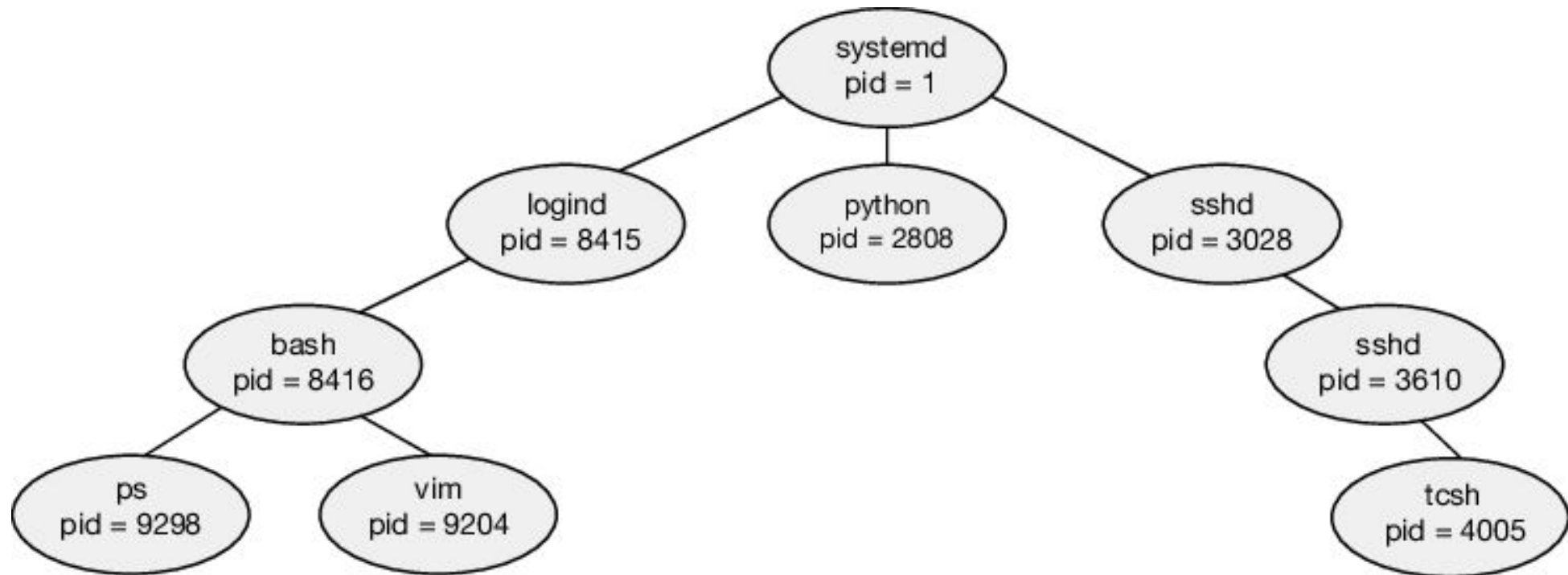
Gerarchia di processi

- **Nella maggior parte dei sistemi operativi**
 - i processi sono organizzati in forma gerarchica
 - quando un processo crea un nuovo processo, il processo creante viene detto padre e il creato figlio
 - si viene così a creare un albero di processi
- **Motivazioni**
 - semplificazione del procedimento di creazione di processi
 - non occorre specificare esplicitamente tutti i parametri e le caratteristiche
 - ciò che non viene specificato, viene ereditato dal padre

Gerarchia di processi: Esempio UNIX



Esempio di albero dei processi di un tipico sistema Linux



Linux: comando ps

```
stefano-ferretti@stefano-ferretti-Precision-T1700: ~  
stefano-ferretti@stefano-ferretti-Precision-T1700:~$ ps -el
```

F	S	UID	PID	PPID	C	PRI	NI	ADDR	SZ	WCHAN	TTY	TIME	CMD
4	S	0	1	0	0	80	0	-	5933	-	?	00:00:02	systemd
1	S	0	2	0	0	80	0	-	0	-	?	00:00:00	kthreadd
1	S	0	3	2	0	80	0	-	0	-	?	00:00:00	pool_workq
1	I	0	4	2	0	60	-20	-	0	-	?	00:00:00	kworker/R-
1	I	0	5	2	0	60	-20	-	0	-	?	00:00:00	kworker/R-
1	I	0	6	2	0	60	-20	-	0	-	?	00:00:00	kworker/R-
1	I	0	7	2	0	60	-20	-	0	-	?	00:00:00	kworker/R-
1	I	0	9	2	0	60	-20	-	0	-	?	00:00:00	kworker/0:
1	I	0	12	2	0	60	-20	-	0	-	?	00:00:00	kworker/R-
1	I	0	13	2	0	80	0	-	0	-	?	00:00:00	rcu_tasks_
1	I	0	14	2	0	80	0	-	0	-	?	00:00:00	rcu_tasks_
1	I	0	15	2	0	80	0	-	0	-	?	00:00:00	rcu_tasks_
1	S	0	16	2	0	80	0	-	0	-	?	00:00:00	ksoftirqd/
1	I	0	17	2	0	80	0	-	0	-	?	00:00:01	rcu_preemp
1	S	0	18	2	0	-40	-	-	0	-	?	00:00:00	migration/
1	S	0	19	2	0	9	-	-	0	-	?	00:00:00	idle_injec
1	S	0	20	2	0	80	0	-	0	-	?	00:00:00	cpuhp/0
5	S	0	21	2	0	80	0	-	0	-	?	00:00:00	cpuhp/1
1	S	0	22	2	0	9	-	-	0	-	?	00:00:00	idle_injec
1	S	0	23	2	0	-40	-	-	0	-	?	00:00:00	migration/
1	S	0	24	2	0	80	0	-	0	-	?	00:00:00	ksoftirqd/
1	I	0	26	2	0	60	-20	-	0	-	?	00:00:00	kworker/1:

Linux: comando top

```
stefano-ferretti@stefano-ferretti-Precision-T1700: ~  
top - 14:58:53 up 46 min, 1 user, load average: 0,48, 0,57, 0,72  
Attività: 297 totale, 1 in esecuzione, 296 dormiente, 0 fermo, 0 zombie  
%Cpu(s): 0,4 us, 0,2 sy, 0,0 ni, 97,5 id, 1,9 wa, 0,0 hi, 0,0 si, 0,0 st  
MiB Mem : 7882,6 totale, 416,5 libero, 3930,2 usato, 3991,9 buff/cache  
MiB Swap: 4096,0 totale, 4092,2 libero, 3,8 usato. 3952,3 disponibile
```

PID	UTENTE	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMANDO
3694	stefano+	20	0	32,8g	431716	253880	S	1,0	5,3	1:55.04	chrome
2599	root	20	0	7837992	129024	85600	S	0,7	1,6	0:28.93	Xorg
646	root	0	-20	0	0	0	I	0,3	0,0	0:01.08	kworker+
1504	root	-51	0	0	0	0	S	0,3	0,0	0:24.54	irq/41-+
3618	stefano+	20	0	3582900	653156	203372	S	0,3	8,1	0:32.38	thunder+
3843	stefano+	20	0	32,8g	283264	157028	S	0,3	3,5	1:19.27	chrome
3859	stefano+	20	0	32,4g	56188	40576	S	0,3	0,7	0:00.85	chrome
16673	stefano+	20	0	1137,4g	434512	139160	S	0,3	5,4	1:11.24	chrome
23926	stefano+	20	0	1135,5g	140852	103468	S	0,3	1,7	0:00.74	chrome
24007	stefano+	20	0	842660	57876	46212	S	0,3	0,7	0:01.36	gnome-t+
1	root	20	0	23732	14168	9432	S	0,0	0,2	0:02.50	systemd
2	root	20	0	0	0	0	S	0,0	0,0	0:00.00	kthreadd
3	root	20	0	0	0	0	S	0,0	0,0	0:00.00	pool_wo+
4	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker+
5	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker+
6	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker+
7	root	0	-20	0	0	0	I	0,0	0,0	0:00.00	kworker+

Linux: comando top

- Il comando top fornisce una panoramica in tempo reale delle risorse di sistema e dei processi in esecuzione
- Lista dei processi:
 - PID: ID del processo.
 - USER: Utente proprietario del processo.
 - PR: Priorità del processo.
 - NI: Valore nice del processo.
 - VIRT: Memoria virtuale utilizzata dal processo.
 - RES: Memoria fisica utilizzata dal processo.
 - SHR: Memoria condivisa utilizzata dal processo.
 - S: Stato del processo (es. S=sleep, R=running, Z=zombie).
 - %CPU: Percentuale di utilizzo della CPU da parte del processo.
 - %MEM: Percentuale di utilizzo della memoria da parte del processo.

Processi e Thread

- **La nozione di processo discussa in precedenza assume che ogni processo abbia una singola “linea di controllo”**
 - per ogni processo, viene eseguite una singola sequenza di istruzioni
 - un singolo processo non può eseguire due differenti attività contemporaneamente
- **Esempi:**
 - scaricamento di due differenti pagine in un web browser
 - inserimento di nuovo testo in un wordprocessor mentre viene eseguito il correttore ortografico

Processi e Thread

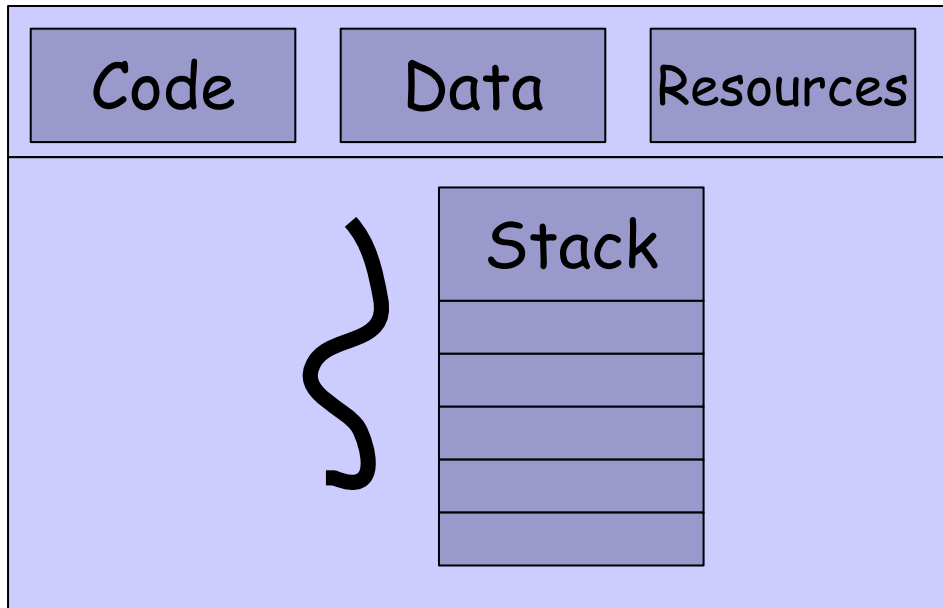
- **Tutti i sistemi operativi moderni**
 - supportano l'esistenza di processi multithreaded
 - in un processo multithreaded esistono molte “linee di controllo”, ognuna delle quali può eseguire un diverso insieme di istruzioni
- **Esempi:**
 - Associando un thread ad ogni finestra aperta in un web browser, è possibile scaricare i dati in modo indipendente

Processi e thread

- **Un thread è l'unità base di utilizzazione della CPU**
- **Ogni thread possiede**
 - la propria copia dello stato del processore
 - il proprio program counter
 - uno stack separato
- **I thread appartenenti allo stesso processo condividono:**
 - codice
 - dati
 - risorse di I/O

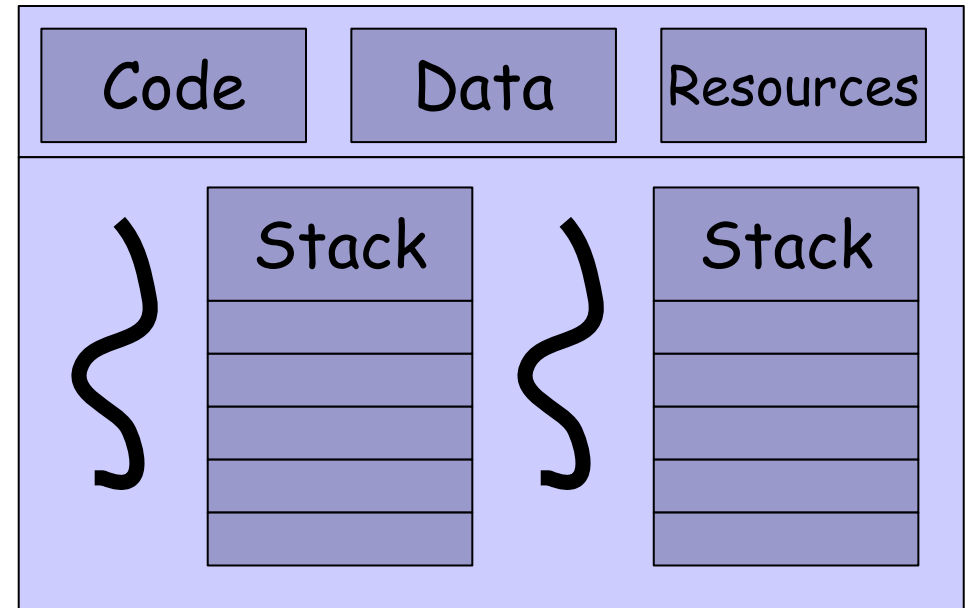
Processi e thread

Processo



Single threaded

Processo



Multi threaded

Benefici dei thread

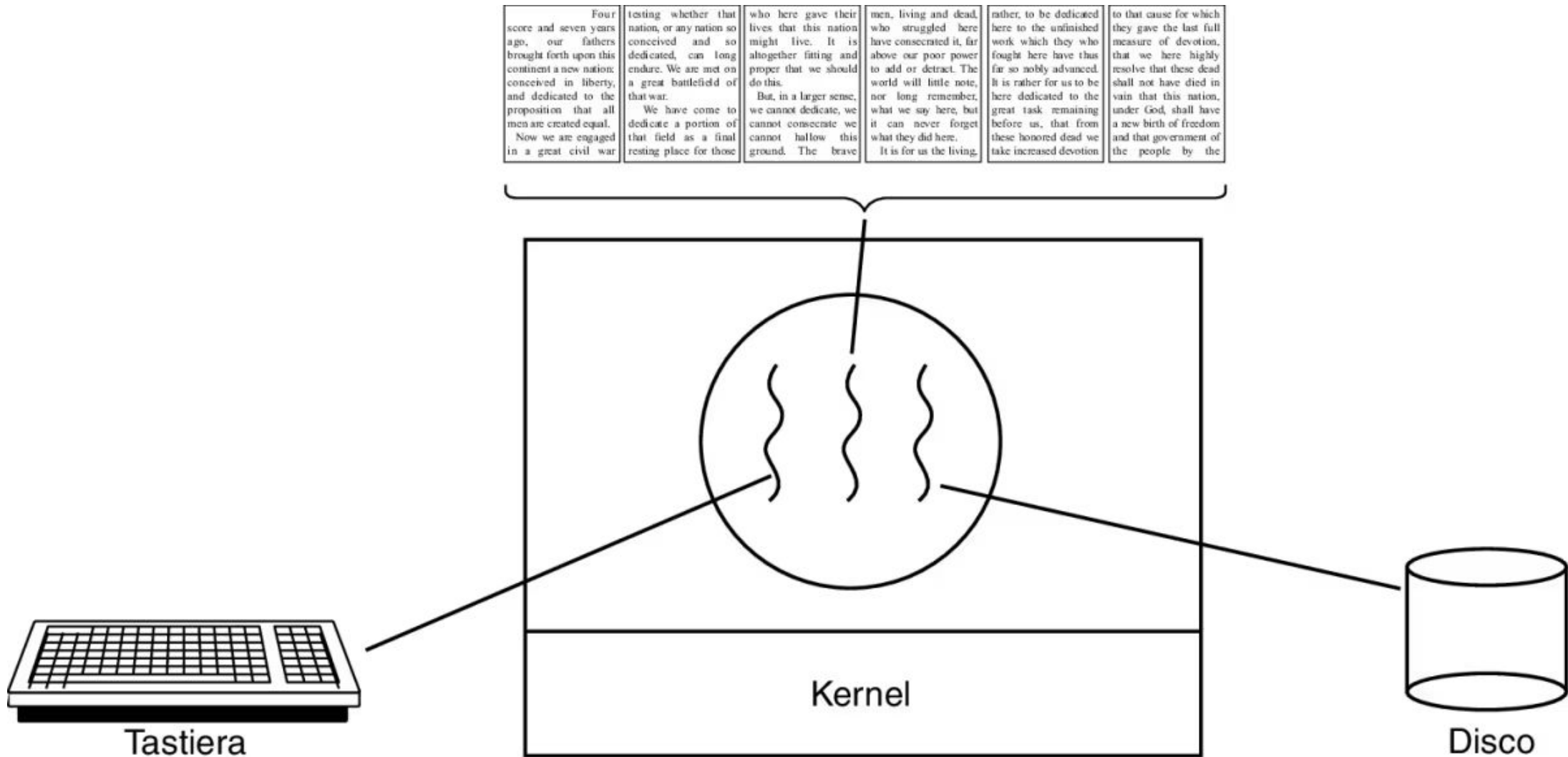
- **Condivisione di risorse**

- i thread condividono lo spazio di memoria e le risorse allocate degli altri thread dello stesso processo
- condividere informazioni tra thread logicamente correlati rende più semplice l'implementazione di certe applicazioni

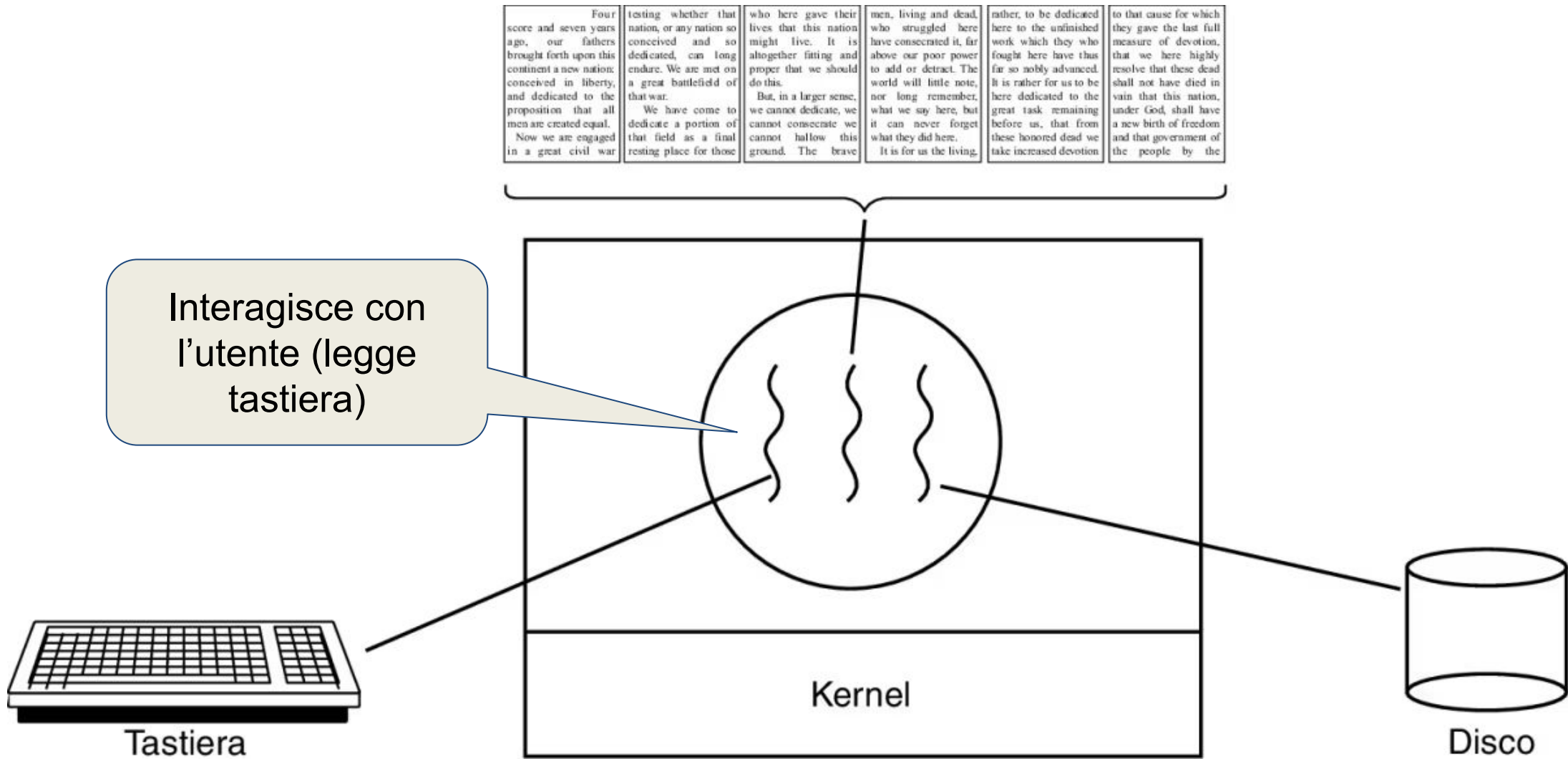
- **Esempio:**

- web browser: condivisione dei parametri di configurazione fra i vari thread

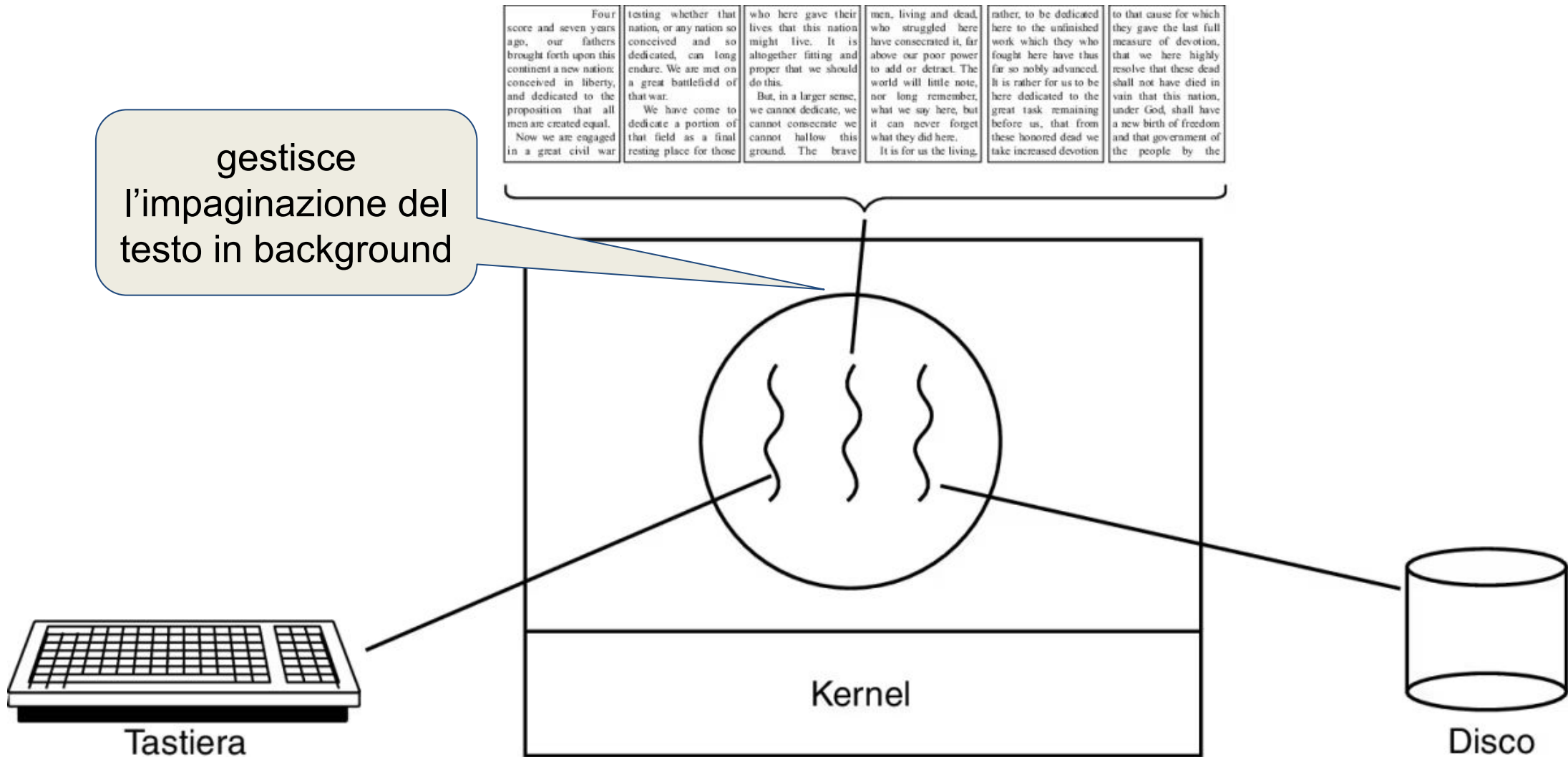
Un elaboratore di testi con 3 thread



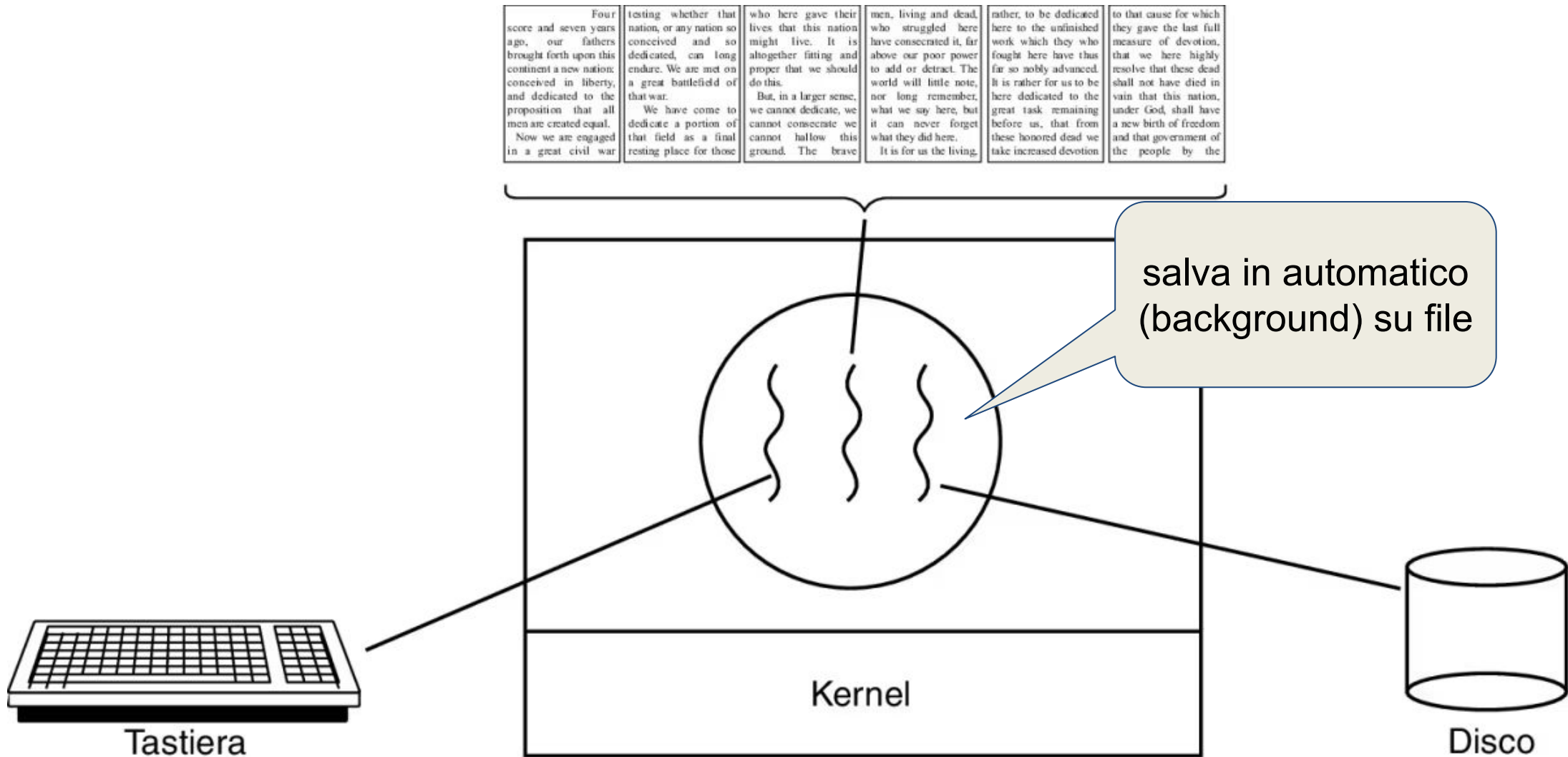
Un elaboratore di testi con 3 thread



Un elaboratore di testi con 3 thread



Un elaboratore di testi con 3 thread



Benefici dei thread

- **Economia**

- allocare memoria e risorse per creare nuovi processi è costoso
- fare context switching fra diversi processi è costoso

- **Gestire i thread è in generale più economico, quindi preferibile**

- creare thread all'interno di un processo è meno costoso
- fare context switching fra thread è meno costoso

- **Esempio:**

- creare un thread in Solaris richiede 1/30 del tempo richiesto per creare un nuovo processo

Processi vs Thread

- Thread = processi "lightweight"
 - utilizzare i thread al posto dei processi rende l'implementazione più efficiente
 - in ogni caso, abbiamo bisogno di processi distinti per applicazioni differenti

Multithreading: implementazione

- Un sistema operativo può implementare i thread in due modi:
 - User thread
(A livello utente)
 - Kernel thread
(A livello kernel)

User thread

- Gli **user thread** vengono supportati sopra il kernel e vengono implementati da una "**thread library**" a livello utente
 - la thread library fornisce supporto per la creazione, lo scheduling e la gestione dei thread senza alcun intervento del kernel
- **Vantaggi:**
 - l'implementazione risultante è molto efficiente
- **Svantaggi:**
 - se il kernel è single-threaded, qualsiasi user thread che effettua una chiamata di sistema bloccante (che si pone in attesa di I/O) causa il blocco dell'intero processo

Kernel thread

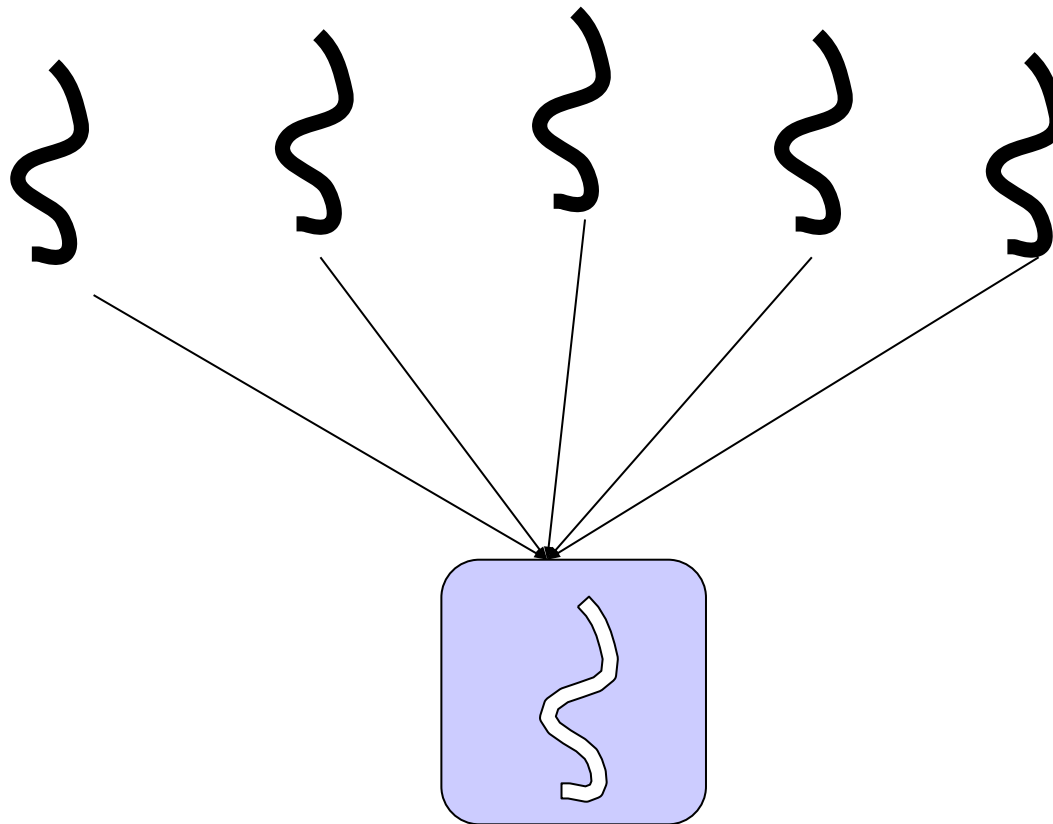
- **I kernel thread vengono supportati direttamente dal sistema operativo**
 - la creazione, lo scheduling e la gestione dei thread sono implementati a livello kernel
- **Vantaggi:**
 - poichè è il kernel a gestire lo scheduling dei thread, se un thread esegue una operazione di I/O, il kernel può selezionare un altro thread in attesa di essere eseguito
- **Svantaggi:**
 - l'implementazione risultante è più lenta, perché richiede un passaggio da livello utente a livello supervisore

Modelli di multithreading

- Molti sistemi supportano sia kernel thread che user thread
- Si vengono così a creare tre differenti modelli di multithreading:
 - Many-to-One
 - One-to-One
 - Many-to-Many

Many-to-One Multithreading

- Un certo numero di user thread vengono mappati su un solo kernel thread
- Modello generalmente adottato da s.o. che non supportano kernel thread multipli

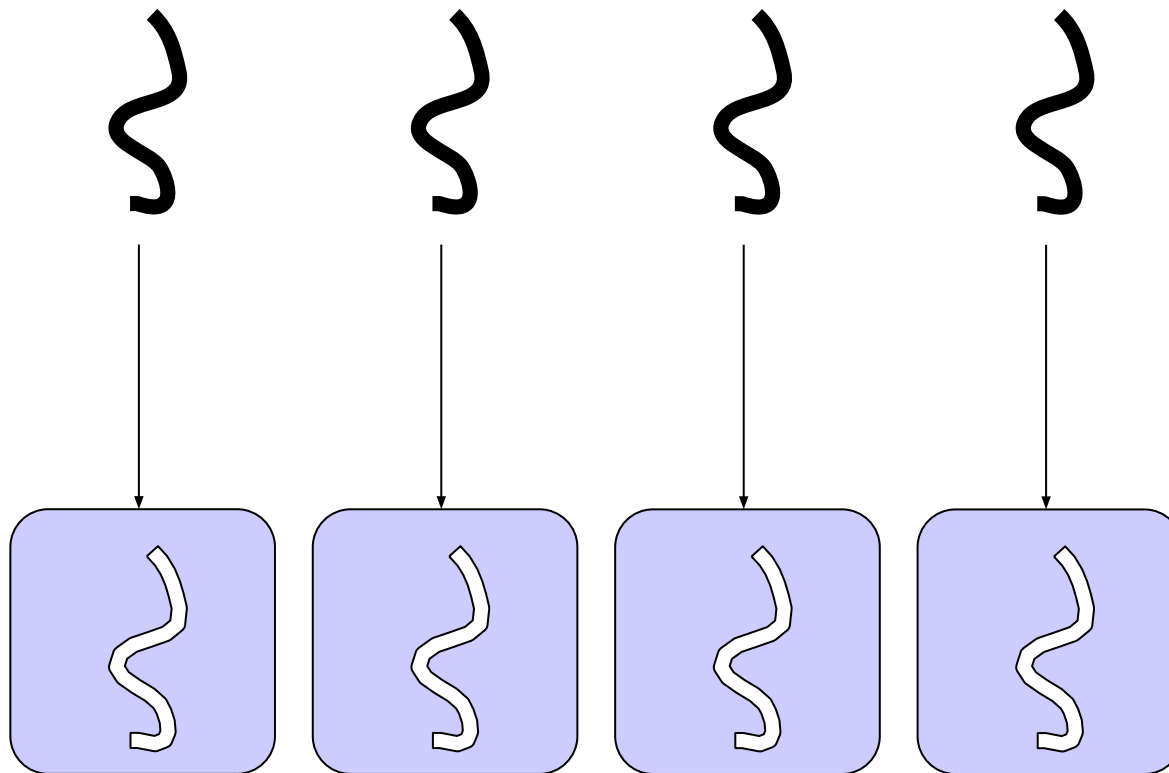


User
Thread(s)

Kernel
Thread(s)

One-to-One Multithreading

- Ogni user thread viene mappato su un kernel thread
- Può creare problemi di scalabilità per il kernel

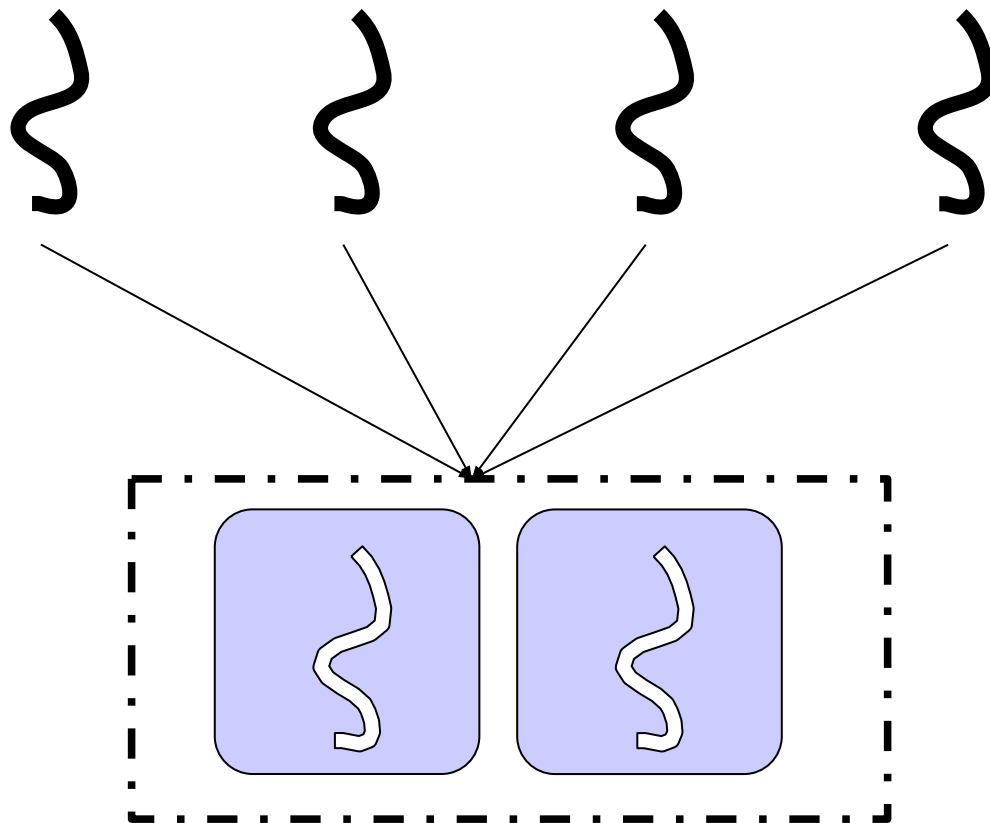


User
Thread(s)

Kernel
Thread(s)

Many-to-Many Multithreading

- Riassume i benefici di entrambe le architetture
- Supportato da Solaris, IRIX, Digital Unix



User
Thread(s)

Kernel
Thread(s)

Processi in Linux

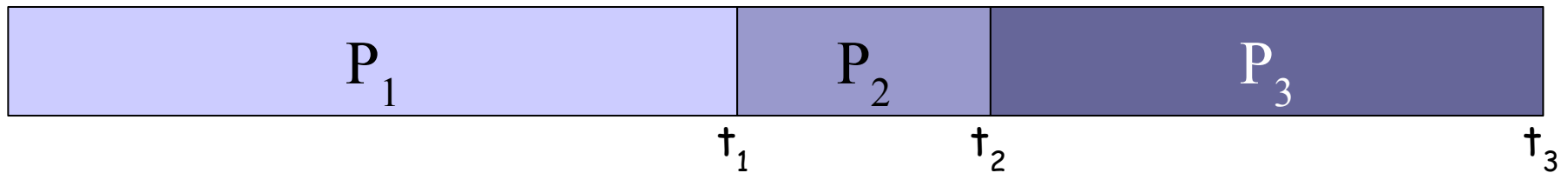
- Un breve approfondimento

2. Scheduling

Rappresentazione degli schedule

- **Diagramma di Gantt**

- per rappresentare uno schedule si usano i diagrammi di Gantt

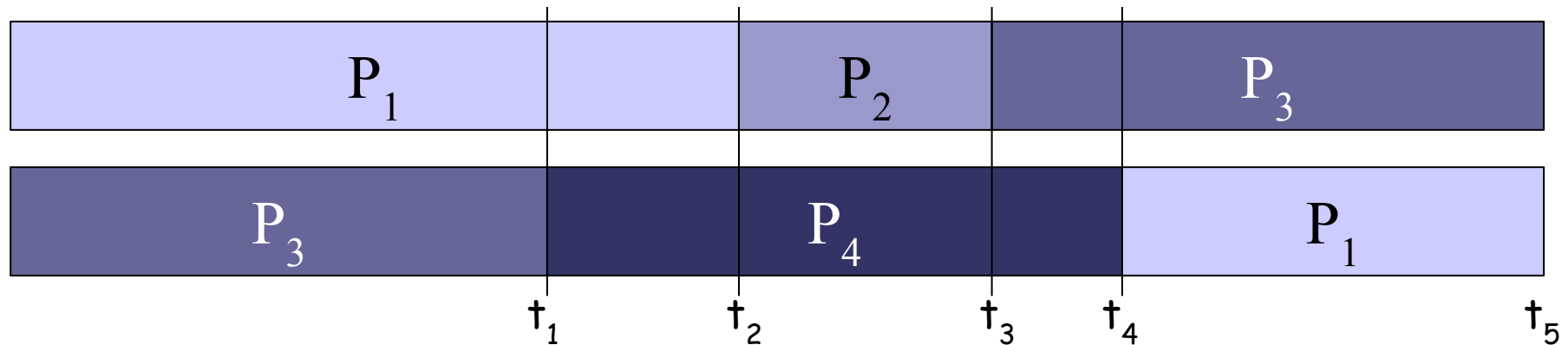


- in questo esempio, la risorsa (es. CPU) viene utilizzata
 - dal processo P_1 dal tempo 0 a t_1 ,
 - viene quindi assegnata a P_2 fino al tempo t_2 e quindi
 - a P_3 fino al tempo t_3

Rappresentazione degli schedule

- **Diagramma di Gantt multi-risorsa**

- nel caso si debba rappresentare lo schedule di più risorse (e.g., un sistema multiprocessore) il diagramma di Gantt risulta composto da più righe parallele



Tipi di scheduler

- **Eventi che possono causare un context switch**

1. quando un processo passa da stato running a stato waiting (system call bloccante, operazione di I/O)
2. quando un processo passa dallo stato running allo stato ready (a causa di un interrupt)
3. quando un processo passa dallo stato waiting allo stato ready
4. quando un processo termina

- **Nota:**

- nelle condizioni 1 e 4, l'unica scelta possibile è quella di selezionare un altro processo per l'esecuzione
- nelle condizioni 2 e 3, è possibile continuare ad eseguire il processo corrente

Tipi di scheduler

- Uno scheduler si dice **non-preemptive** o **cooperativo**
 - se i context switch avvengono solo nelle condizioni 1 e 4
 - in altre parole: *il controllo della risorsa viene trasferito solo se l'assegnatario attuale lo cede volontariamente*
 - Windows 3.1, Mac OS y con $y < 8$
- Uno scheduler si dice **preemptive** se
 - se i context switch possono avvenire in ogni condizione
 - in altre parole: *è possibile che il controllo della risorsa venga tolto all'assegnatario attuale a causa di un evento*
 - tutti gli scheduler moderni

Tipi di scheduler

- **Vantaggi dello scheduling cooperativo**
 - non richiede alcuni meccanismi hardware come ad esempio timer programmabili
- **Vantaggi dello scheduling preemptive**
 - permette di utilizzare al meglio le risorse

Criteri di scelta di uno scheduler

- **Utilizzo della risorsa (CPU)**

- percentuale di tempo in cui la CPU è occupata ad eseguire processi
- deve essere massimizzato

- **Throughput**

- numero di processi completati per unità di tempo
- dipende dalla lunghezza dei processi
- deve essere massimizzato

- **Tempo di turnaround**

- tempo che intercorre dalla sottomissione di un processo alla sua terminazione
- deve essere minimizzato

Criteri di scelta di uno scheduler

- **Tempo di attesa**

- il tempo trascorso da un processo nella coda ready
- deve essere minimizzato

- **Tempo di risposta**

- tempo che intercorre fra la sottomissione di un processo e il tempo di prima risposta
- particolarmente significativo nei programmi interattivi, deve essere minimizzato

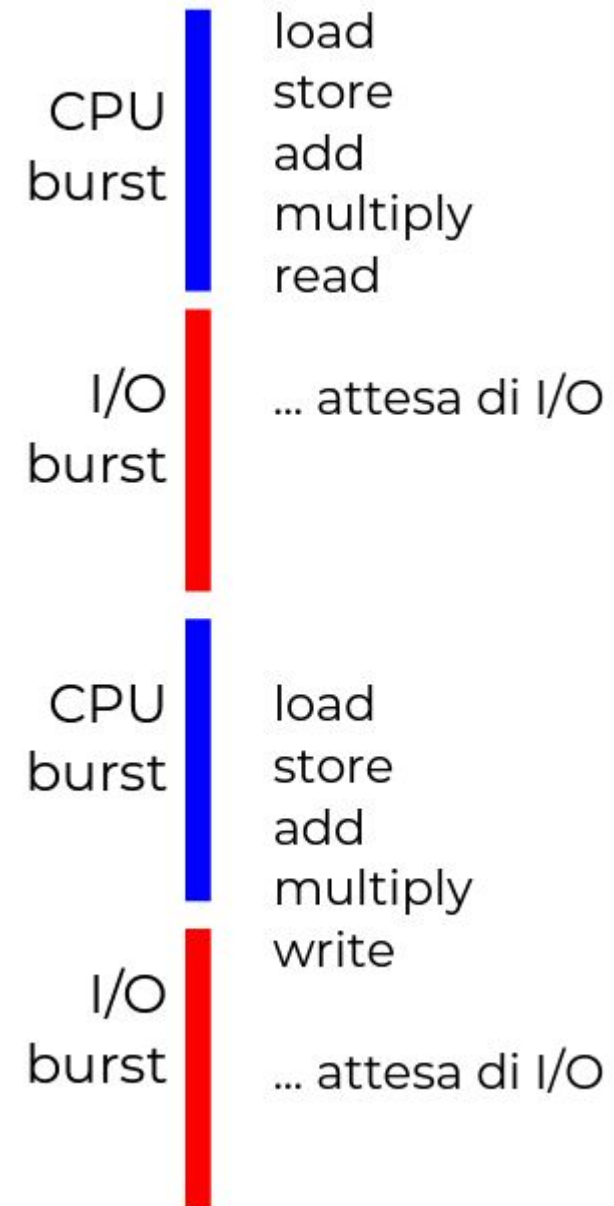
Caratteristiche dei processi

- Durante l'esecuzione di un processo:

- si alternano periodi di attività svolte dalla CPU (*CPU burst*)...
- ...e periodi di attività di I/O (*I/O burst*)

- I processi:

- caratterizzati da CPU burst molto lunghi si dicono *CPU bound*
- caratterizzati da I/O burst molto lunghi si dicono *I/O bound*



Scheduling

- **Algoritmi:**
 - First Come, First Served
 - Shortest-Job First
 - Shortest-Next-CPU-Burst First
 - Shortest-Remaining-Time-First
 - Round-Robin

First Come, First Served (FCFS)

- **Algoritmo**

- il processo che arriva per primo, viene servito per primo
- politica senza preemption

- **Implementazione**

- semplice, tramite una coda (politica FIFO)

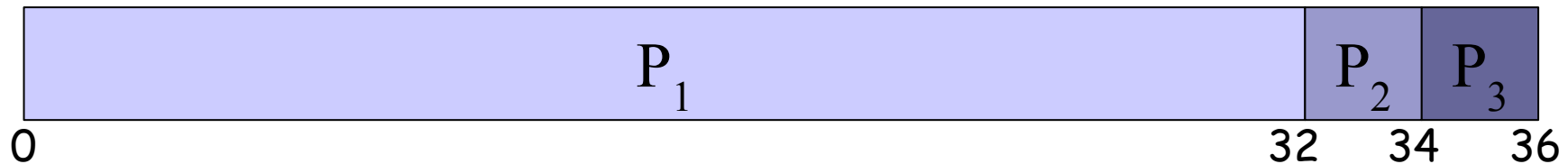
- **Problemi**

- elevati tempi medi di attesa e di turnaround
- processi CPU bound ritardano i processi I/O bound

First Come, First Served (FCFS)

- **Esempio:**

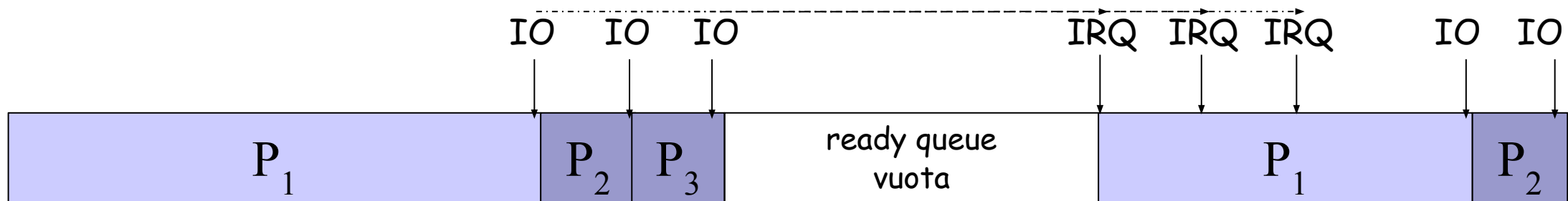
- ordine di arrivo: P_1, P_2, P_3
- lunghezza dei CPU-burst in ms: 32, 2, 2
- Tempo medio di turnaround
 - $(32+34+36)/3 = 34$ ms
- Tempo medio di attesa:
 - $(0+32+34)/3 = 22$ ms



First Come, First Served (FCFS)

- Supponiamo di avere

- un processo CPU bound
- un certo numero di processi I/O bound
- i processi I/O bound si "mettono in coda" dietro al processo CPU bound, e in alcuni casi la ready queue si può svuotare
- *Convoy effect*



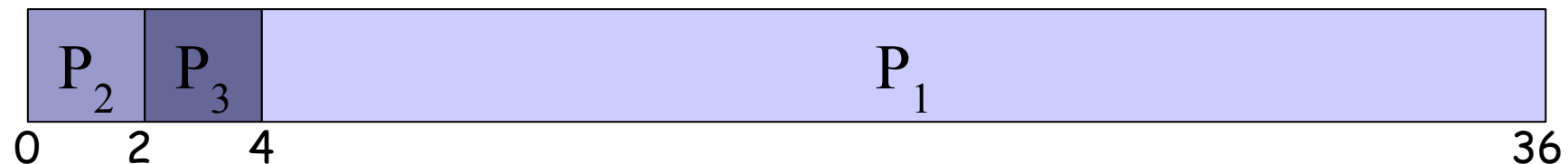
Shortest Job First (SJF)

- **Algoritmo (Shortest Next CPU Burst First)**

- la CPU viene assegnata al processo ready che ha la minima durata del CPU burst successivo
- politica senza preemption

- **Esempio**

- Tempo medio di turnaround: $(0+2+4+36)/3 = 7$ ms
- Tempo medio di attesa: $(0+2+4)/3 = 2$ ms



Shortest Job First (SJF)

- **L'algoritmo SJF**

- è *ottimale* rispetto al tempo di attesa, in quanto è possibile dimostrare che produce il minor tempo di attesa possibile
- ma è *impossibile da implementare* in pratica!
- è possibile solo fornire delle *approssimazioni*

- **Negli scheduler long-term**

- possiamo chiedere a chi sottomette un job di predire la durata del job

- **Negli scheduler short-term**

- non possiamo conoscere la lunghezza del *prossimo* CPU burst.... ma conosciamo la lunghezza di quelli *precedenti*

Shortest Job First (SJF)

- **Calcolo approssimato della durata del CPU burst**

- basata su *media esponenziale* dei CPU burst precedenti
- sia t_n il tempo dell' n -esimo CPU burst e τ_n la corrispondente previsione;
 τ_{n+1} può essere calcolato come segue:

$$\tau_{n+1} = \alpha t_n + (1-\alpha)\tau_n$$

- **Media esponenziale**

- svolgendo la formula di ricorrenza, si ottiene

$$\tau_{n+1} = \sum_{j=0..n} \alpha(1-\alpha)^j t_{n-j} + (1-\alpha)^{n+1} \tau_0$$

da cui il nome media esponenziale

Shortest Job First (SJF)

- **Spiegazione**

- t_n rappresenta la storia recente
- τ_n rappresenta la storia passata
- α rappresenta il peso relativo di storia passata e recente
- cosa succede con $\alpha = 0$, 1 oppure $\frac{1}{2}$?

- **Nota importante:**

- SJF può essere soggetto a starvation!

Shortest Job First (SJF)

- Shortest Job First "approssimato" esiste in due versioni:
 - *non preemptive*
 - il processo corrente esegue fino al completamento del suo CPU burst
 - *preemptive*
 - il processo corrente può essere messo nella coda ready, se arriva un processo con un CPU burst più breve di quanto rimane da eseguire al processo corrente
 - "Shortest-Remaining-Time First"

Scheduling Round-Robin

- **E' basato sul concetto di quanto di tempo (o time slice)**
 - un processo non può rimanere in esecuzione per un tempo superiore alla durata del quanto di tempo
- **Implementazione (1)**
 - l'insieme dei processi pronti è organizzato come una coda
 - due possibilità:
 - un processo può lasciare il processore *volontariamente*, in seguito ad un'operazione di I/O
 - un processo può *esaurire il suo quanto di tempo* senza completare il suo CPU burst, nel qual caso viene aggiunto in fondo alla coda dei processi pronti
 - in entrambi i casi, il prossimo processo da eseguire è il primo della coda dei processi pronti

Scheduling Round-Robin

- **La durata del quanto di tempo è un parametro critico del sistema**
 - se il quanto di tempo è breve, il sistema è meno efficiente perchè deve cambiare il processo attivo più spesso
 - se il quanto è lungo, in presenza di numerosi processi pronti ci sono lunghi periodi di inattività di ogni singolo processo,
 - in sistemi interattivi, questo può essere fastidioso per gli utenti

Scheduling Round-Robin

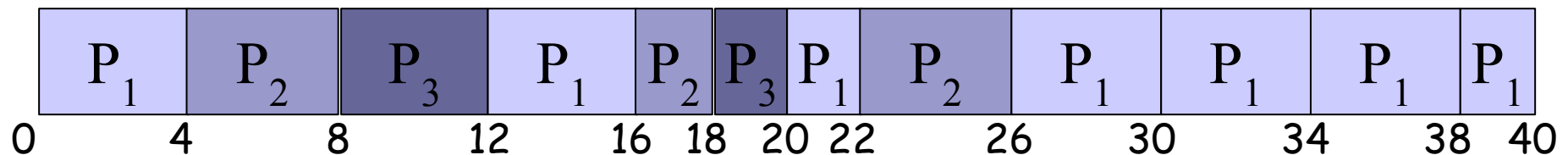
- **Implementazione (2)**

- è necessario che l'hardware fornisca un timer (interval timer) che agisca come "sveglia" del processore
- il timer è un dispositivo che, attivato con un preciso valore di tempo, è in grado di fornire un interrupt allo scadere del tempo prefissato
- il timer viene interfacciato come se fosse un'unità di I/O

Scheduling Round-Robin

• Esempio

- tre processi P_1 , P_2 , P_3
- lunghezza dei CPU-burst in ms (P_1 : 10+14; P_2 : 6+4; P_3 : 6)
- lunghezza del quanto di tempo: 4
- Tempo medio di turnaround $(40+26+20)/3 = 28.66$ ms
- Tempo medio di attesa: $(16+16+14)/3 = 15.33$ ms
 - NB (supponiamo attese di I/O brevi, < 2 ms)
- Tempo medio di risposta: 4 ms



Un po' di simulazione ...

- vedi [process_scheduling_simulation.c](#)

Scheduling a priorità

- **Round-robin**

- fornisce le stesse possibilità di esecuzione a tutti i processi

- **Ma i processi non sono tutti uguali:**

- usando round-robin puro la visualizzazione di un video MPEG potrebbe essere ritardata da un processo che sta smistando la posta
- la lettera può aspettare $\frac{1}{2}$ sec, il frame video NO

Scheduling a priorità

- **Descrizione**

- a ogni processo è associata una specifica priorità
- lo scheduler sceglie il processo pronto con priorità più alta

- **Le priorità possono essere:**

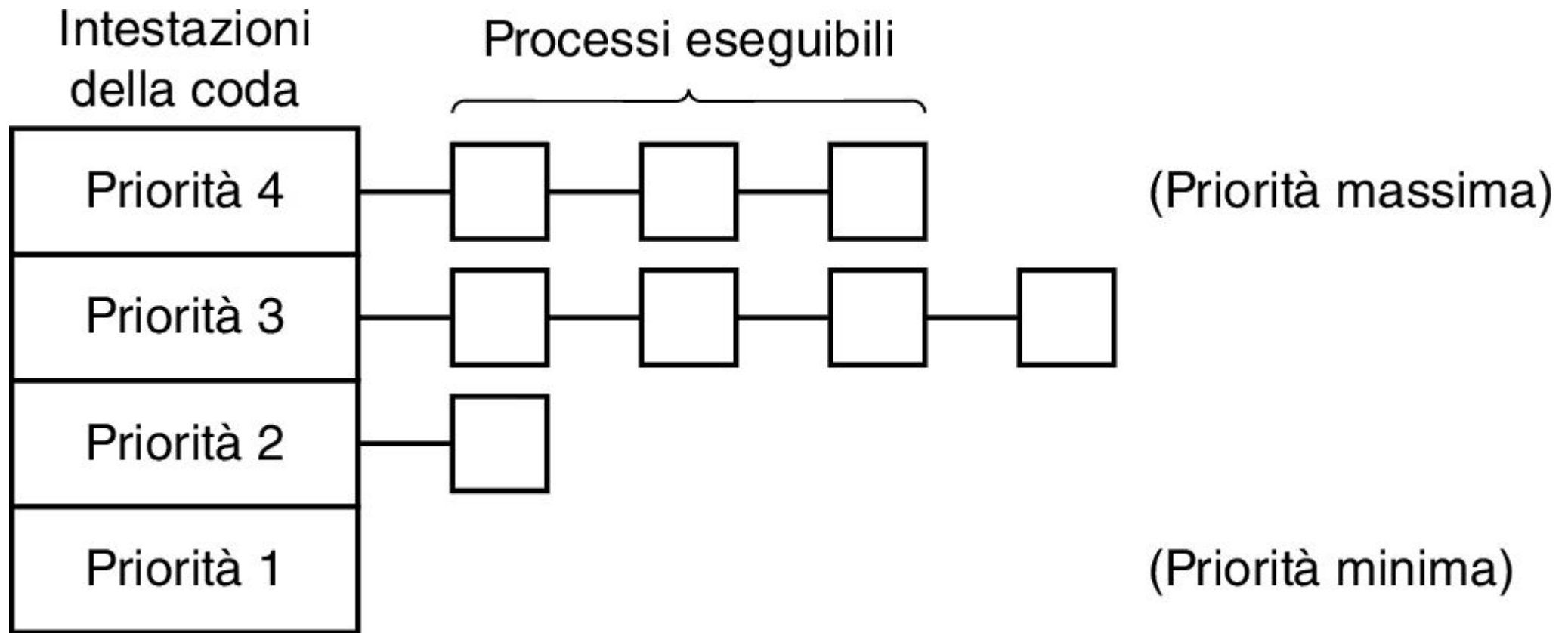
- *definite dal sistema operativo*

- vengono utilizzate una o più quantità misurabili per calcolare la priorità di un processo
- esempio: SJF è un sistema basato su priorità

- *definite esternamente*

- le priorità non vengono definite dal sistema operativo, ma vengono imposte dal livello utente

Scheduling a priorità



Scheduling a priorità

- **Priorità statica**

- la priorità non cambia durante la vita di un processo
- problema: processi a bassa priorità possono essere posti in *starvation* da processi ad alta priorità

- **Priorità dinamica**

- la priorità può variare durante la vita di un processo
- è possibile utilizzare metodologie di priorità dinamica per evitare *starvation*

Scheduling a priorità

- **Priorità basata su aging**

- tecnica che consiste nell'incrementare gradualmente la priorità dei processi in attesa
- posto che il range di variazione delle priorità sia limitato, nessun processo rimarrà in attesa per un tempo indefinito perché prima o poi raggiungerà la priorità massima

Scheduling a classi di priorità

- **Descrizione**

- è possibile creare diverse classi di processi con caratteristiche simili e assegnare ad ogni classe specifiche priorità
- la coda ready viene quindi scomposta in molteplici "sottocode", una per ogni classe di processi

- **Algoritmo**

- uno scheduler a classi di priorità seleziona il processo da eseguire fra quelli pronti della classe a priorità massima che contiene processi

Scheduling Multilivello

- **Descrizione**

- all'interno di ogni classe di processi, è possibile utilizzare una politica specifica adatta alle caratteristiche della classe
- uno scheduler multilivello cerca prima la classe di priorità massima che ha almeno un processo ready
- sceglie poi il processo da porre in stato running coerentemente con la politica specifica della classe

Scheduling Multilivello - Esempio

- **Quattro classi di processi (priorità decrescente)**
 - processi server (priorità statica)
 - processi utente interattivi (round-robin)
 - altri processi utente (FIFO)
 - il processo vuoto (FIFO banale)

3. Scheduling Real-Time

Scheduling Real-Time

- **In un sistema real-time**

- la correttezza dell'esecuzione non dipende solamente dal valore del risultato, ma anche dall'istante temporale nel quale il risultato viene emesso

- **Hard real-time**

- le deadline di esecuzione dei programmi non devono essere superate in nessun caso
- sistemi di controllo nei velivoli, centrali nucleari o per la cura intensiva dei malati

- **Soft real-time**

- errori occasionali sono tollerabili
- ricostruzione di segnali audio-video, transazioni interattive

Scheduling Real-Time

- **Processi periodici**

- sono periodici i processi che vengono riattivati con una cadenza regolare (periodo)
- esempi: controllo assetto dei velivoli, basato su rilevazione periodica dei parametri di volo

- **Processi aperiodici**

- i processi che vengono scatenati da un evento sporadico, ad esempio l'allarme di un rilevatore di pericolo

Esempi di Scheduler RT: Rate Monotonic

- **Descrizione**

- è una politica di scheduling, valida alle seguenti condizioni
 - ogni processo periodico deve completare entro il suo periodo
 - tutti i processi sono indipendenti
 - la preemption avviene istantaneamente e senza overhead
- viene assegnata staticamente una priorità a ogni processo
- processi con frequenza più alta (i.e. periodo più corto) hanno priorità più alta
- ad ogni istante, viene eseguito il processo con priorità più alta (facendo preemption se necessario)

Esempi di Scheduler RT: Earliest Deadline First (EDF)

- **Descrizione**

- è una politica di scheduling per processi periodici real-time
- viene scelto di volta in volta il processo che ha la deadline più prossima
- viene detto "a priorità dinamica" perchè la priorità relativa di due processi varia in momenti diversi